《算法设计与分析》实验报告

实验三　贪心算法

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2024/04/27
班级	22 级计算机 2 班		

实验内容

[要求先在 OJ 上提交通过]

1、[OJ]贪心算法 1

2、[OJ]贪心算法 2

3、[OJ]贪心算法-堂练

实验目的

1）掌握贪心算法的应用

2）学习掌握本类算法的设计与实现，在 OJ 上提交算法代码并测试通过，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

OJ 实验 1 作业正常时间完成。

OJ 实验 2 作业正常时间完成。

OJ 实验 3 堂练全部完成。

实验过程和步骤

Module 1: 贪心算法 1

【1、 减肥（1-1000）】

减肥

Description

小K没事干，他要搬砖头，为了达到较好的减肥效果，教练规定的方式很特别：

每一次，小K可以把两堆砖头合并到一起，消耗的体力等于两堆砖头的重量之和。

经过 n-1次合并后， 就只剩下一堆了。小K在搬砖头时总共消耗的体力等于每次合并所耗体力之和。小K为了偷懒，希望耗费的体力最小。

例如有3堆砖头，数目依次为 1,2,9 . 可以先将 1,2 堆合并，新堆数目为3 ，耗费体力为 3 . 接着, 将新堆与原先的第三堆合并，又得到新的堆，数目为 12 ，耗费体力为12 。所以总共耗费体力 =3+12=15。可以证明15为最小的体力耗费值。

Input

共两行。

第一行是一个整数 n(1≤n≤1000) ，表示砖头堆数。

第二行n个整数，每个整数表示每堆砖头的砖头块数。

Output

一个整数，也就是最小的体力耗费值。

Sample Input 1

```
3
1 2 9
```

Sample Output 1

```
15
```

【2、程序代码及注释】

```cpp
#include<bits/stdc++.h>
using namespace std;

int n;

// 求解函数，参数是数组指针和元素个数
int Solve(int *a, int n) {
    int sum = 0; // 记录最终的总和
    sort(a, a + n); // 先将数组整体排序

    for (int i = 1; i < n; i++) {
        a[i] = a[i-1] + a[i]; // 将前两个元素合并
        sum += a[i]; // 累加合并后的值到总和
        sort(a + i, a + n); // 对剩余部分重新排序
    }

    return sum;
}

int main() {
    int n;
    cin >> n; // 输入元素个数
    int a[n+1] = {0}; // 定义数组，初始化为0
```

```
24.
25.     for (int i = 0; i < n; i++) {
26.         cin >> a[i]; // 输入数组元素
27.     }
28.
29.     cout << Solve(a, n) << endl; // 输出结果
30.     return 0;
31. }
```

【3、遇到的问题及解决方法】

无

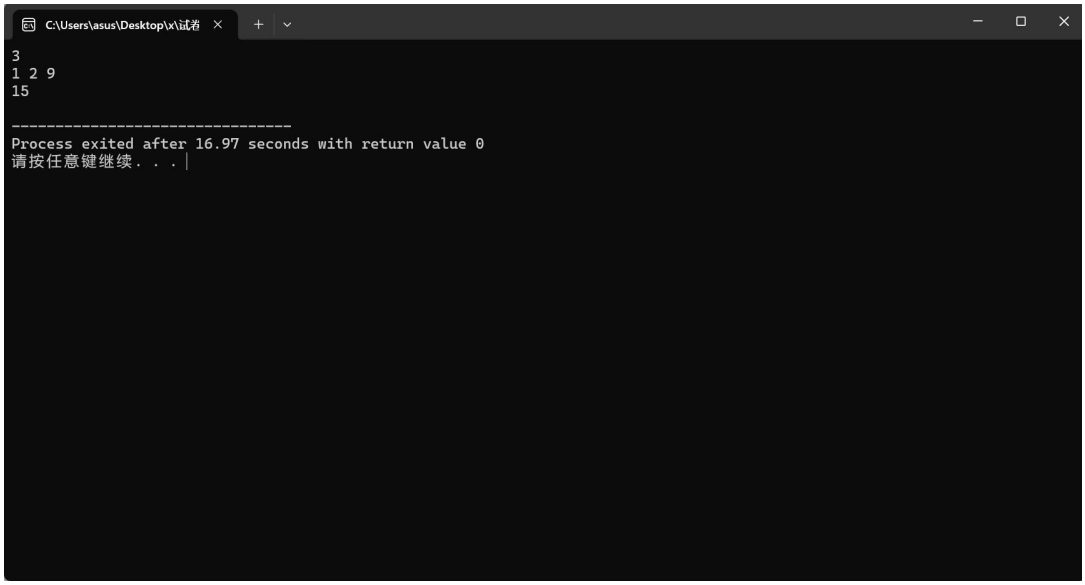
【4、算法分析与知识点】

根据题目描述，要使得搬运砖块数目所耗费体力最少，那么每次搬运时都应该搬运所需体力最少的两块砖。这里指的所需体力最少的两块砖不仅指原始的砖块，还指搬运时堆叠的砖块集合。所以首先我们对砖块由从小到大进行排序，搬运其中最小的两块砖。然后将搬运完成后堆叠的砖块集合放入正确的位置，并将耗费体力计入总数。索引值加一，重复上述操作，直到进行 n-1 次搬砖将全部砖块堆叠完成，我们最终能够得到所需耗费体力的最小值。

另外，也可以用最小堆的算法，每次可以重建堆，自动输出最小值。

算法的时间复杂度为 O（N），空间复杂度也为 O（N）

【5、AC 截图及运行测试截图】



【1、　排队接水（1-1001）】

排队接水

Description

夏天到了，又到了用水高峰期，偏巧小区的水管出了点问题，消防车赶紧给小区送了一车水过来。小区居民们纷纷拿出自家装水的容器，有的是个大塑料瓶，有的是茶水壶，有的是小塑料桶，哈哈，什么样的都有:)。现在有n个人在一个水龙头前排队接水，假设每个人接水的时间分别为Ti，请编程找出这n个人排队的一种顺序，使得这n个人的平均等待时间最小。

Input

输入有多组测试数据

每组测试数据共两行，第一行为一个整数n，表示有n个人，n<=1000;

第二行分别表示第1个人到第n个人每人的接水时间T1,T2,...Tn.

Output

输出文件有两行，第一行为一种排队顺序，即编号从1到n的n个人的一种排序方式;

第二行为这种排序方案下的平均等待时间（输出结果精确到小数点后两位）.

Sample Input 1

```
10
56 12 1 99 1000 234 33 55 99 812
```

Sample Output 1

```
3 2 7 8 1 4 9 6 10 5
291.90
```

【2、程序代码及注释】

```
#include <bits/stdc++.h>
using namespace std;

// 定义一个people类，存储每个人的编号和接水时间
class people {
public:
    int id;     // 人的编号
    int time;   // 接水所需时间
};

// 自定义排序规则：按接水时间time升序排列
bool compare(const people& p1, const people& p2) {
    return p1.time < p2.time;
}

int main() {
    int n;
    // 支持多组测试数据
    while (cin >> n) {
        long sum = 0;     // 累加所有人的等待时间
        int wait = 0;     // 当前累积的等待时间
        vector<people> a(n);  // 存储n个人的信息

        // 读入每个人的接水时间，并赋编号
        for (int i = 0; i < n; i++) {
            cin >> a[i].time;
            a[i].id = i + 1;  // 编号从1开始
        }

        // 按接水时间从小到大排序
```

```
        sort(a.begin(), a.end(), compare);

        // 计算总等待时间并输出排序后的编号
        for (int i = 0; i < n; i++) {
            sum += wait;            // 等待时间累加
            wait += a[i].time;      // 当前人接水完成后，增加等待时间
            cout << a[i].id << " "; // 输出当前排队人的编号
        }
        cout << endl;

        // 输出平均等待时间，保留两位小数
        printf("%.2f\n", 1.0 * sum / n);
    }
}
```

【3、遇到的问题及解决方法】

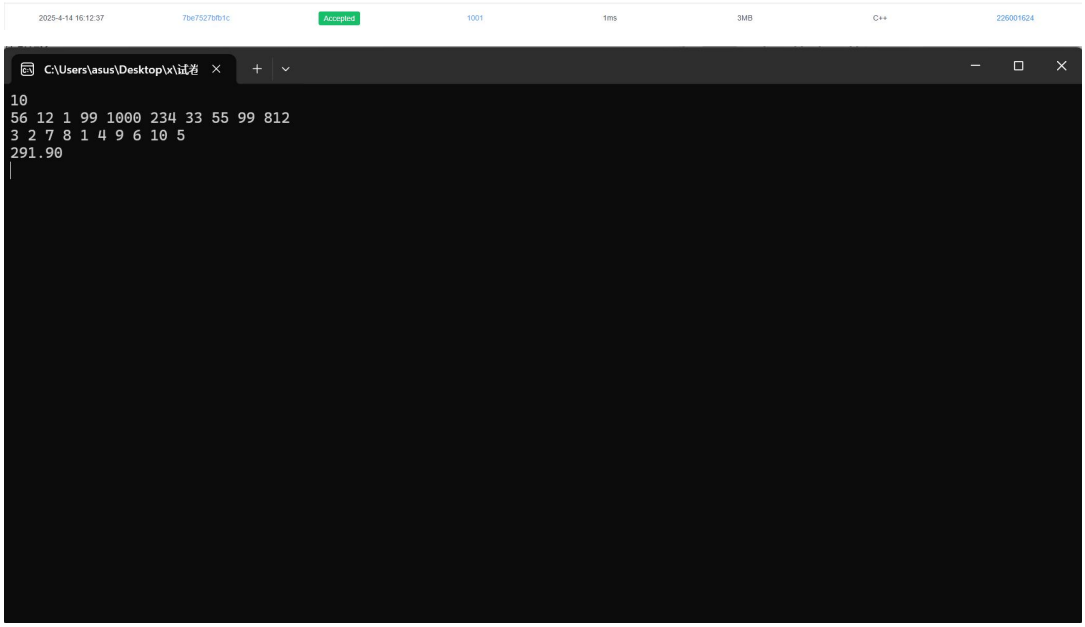
在使用 sort 函数的时候，我错误的使用了<=符号，这就使得 sort 的排序产生了未定义的行为，正确的做法应该是使用<，其余的部分由 compare 函数自己去生成。

【4、算法分析与知识点】

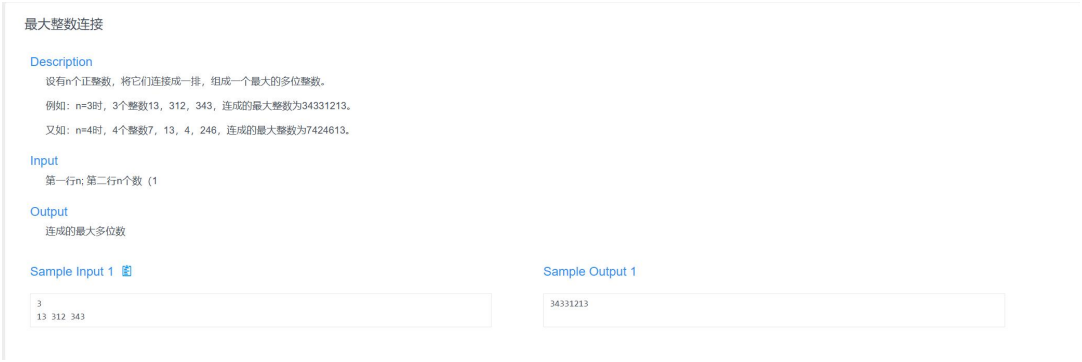
为了最小化所有人的平均等待时间，应当将接水时间较短的人排在队伍前面。这样可以减少后续所有人的等待累积。具体做法是：首先记录每个人的接水时间和编号，然后按照接水时间升序排序，依次安排接水，同时累加等待时间，最后计算出平均等待时间。因为等待时间是累加前面所有人的接水时间，所以时间短的人越早接水，总等待时间越小。

算法的时间复杂度为 O（N），空间复杂度也为 O（N）

【5、AC 截图及运行测试截图】



【1、 最大整数连接（1-1002）】



最大整数连接

Description

设有n个正整数，将它们连接成一排，组成一个最大的多位整数。

例如：n=3时，3个整数13，312，343，连成的最大整数为34331213。

又如：n=4时，4个整数7，13，4，246，连成的最大整数为7424613。

Input

第一行n; 第二行n个数（1

Output

连成的最大多位数

Sample Input 1

```
3
13 312 343
```

Sample Output 1

```
34331213
```

【2、程序代码及注释】

```cpp
#include<bits/stdc++.h>
using namespace std;

// 比较函数：决定排序顺序
// 如果 a+b 字典序 > b+a，则认为 a 应该排在 b 前面
bool Compare(string a, string b) {
    return (a + b) > (b + a);
}

// 找到能够组成的最小数字（通过特定排序）
void FindMin(string a[], int n) {
    // 使用自定义比较函数进行排序
 sort(a, a + n, Compare);

    // 排序后直接输出所有字符串连接成的结果
 for (int i = 0; i < n; i++) {
        cout << a[i];
    }
    cout << endl;
}

int main() {
 int n = 0;
 cin >> n;  // 输入字符串个数

    // 定义一个数组存储字符串，最多1000个
 string a[1000 + 1];

    // 读入所有字符串
 for (int i = 0; i < n; i++) {
  cin >> a[i];
```

```
32. }
33.
34.     // 调用函数输出结果
35. FindMin(a, n);
36. return 0;
37.}
```

【3、遇到的问题及解决方法】

Compole Error:在 OJ 的编译器中。To_string 方法无法使用，故直接以字符串的形式进行输入。

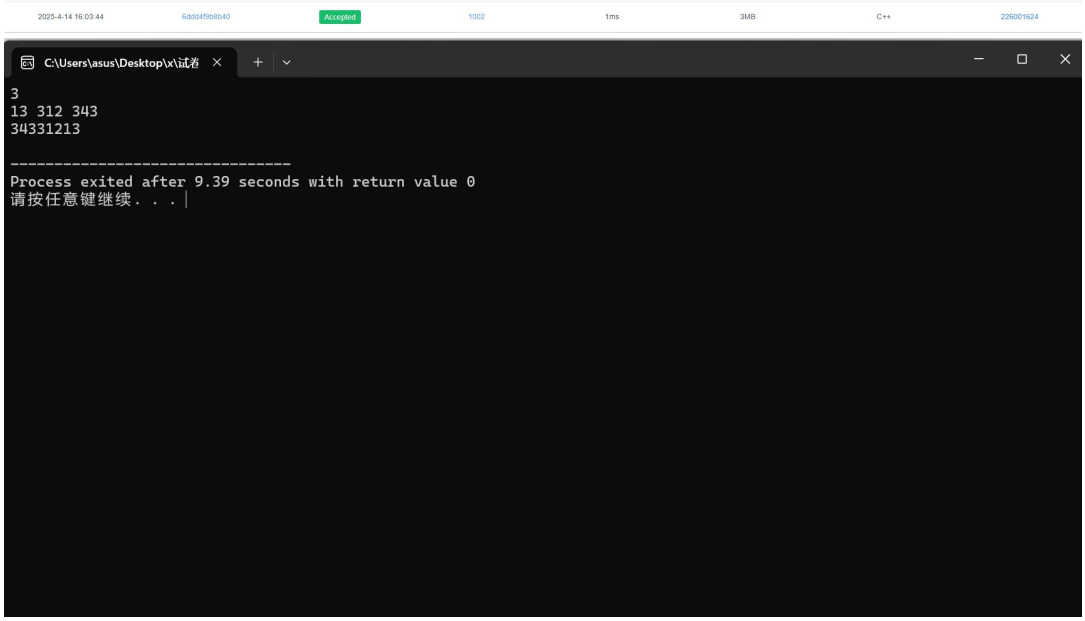
【4、算法分析与知识点】

根据题目描述，本实验需要连接数字为最大。故我们采用贪心算法，在每次连接时都选择连接结果最大的两数。采用自定义 bool Compare(string a, string b) 函数进行比较，sort(a,a+n,Compare)排列，最后以此输出排列完成后的数组即可。

程序的时间复杂度主要由排序过程决定。排序 n 个元素需要 O(n log n) 次比较，每次比较时需要连接两个字符串，连接操作耗时约为 O(L)（其中 L 为字符串平均长度），因此整体时间复杂度为 O(n log n × L)。排序完成后，将所有字符串依次输出，时间复杂度为 O(nL)，但这部分相较于排序时间可以忽略。

程序的空间复杂度主要来源于保存字符串的数组 a，占用 O(nL) 空间。同时，快速排序过程中需要递归调用栈空间约为 O(log n)，总体空间复杂度为 O(nL)。

【5、AC 截图及运行测试截图】



Module 2: 贪心算法 2

【1、 最小跳数（2-1000）】



最小跳数

Description

给定一个非负整数数组，假定你的初始位置为数组第一个位置。数组中的每个元素代表你在那个位置能够跳跃的最大长度。你的目标是到达最后一个下标位置，并且使用最少的跳跃次数。

Input

输入一组非负整数数组，数组长度不超过500。

Output

最少经过几次跳跃，可以到达最后一个位置。

Sample Input 1

```
2 3 1 1 4
```

Sample Output 1

```
2
```

【2、程序代码及注释】

```cpp
#include<bits/stdc++.h>
using namespace std;

int main() {
    int a[501] = {0};
    int n = 0;

    // 读入所有元素，直到输入结束
    while (cin >> a[n++]);
    n--;

    int maxpos = 0, end = 0, step = 0;

    // 遍历到倒数第二个元素
    for (int i = 0; i < n - 1; i++) {
        if (maxpos >= i) {
            maxpos = max(maxpos, i + a[i]);
            if (i == end) { // 到达当前跳跃终点，必须跳跃
                end = maxpos;
                step++;
            }
        }
    }

    cout << step << endl;
}
```

【3、遇到的问题及解决方法】

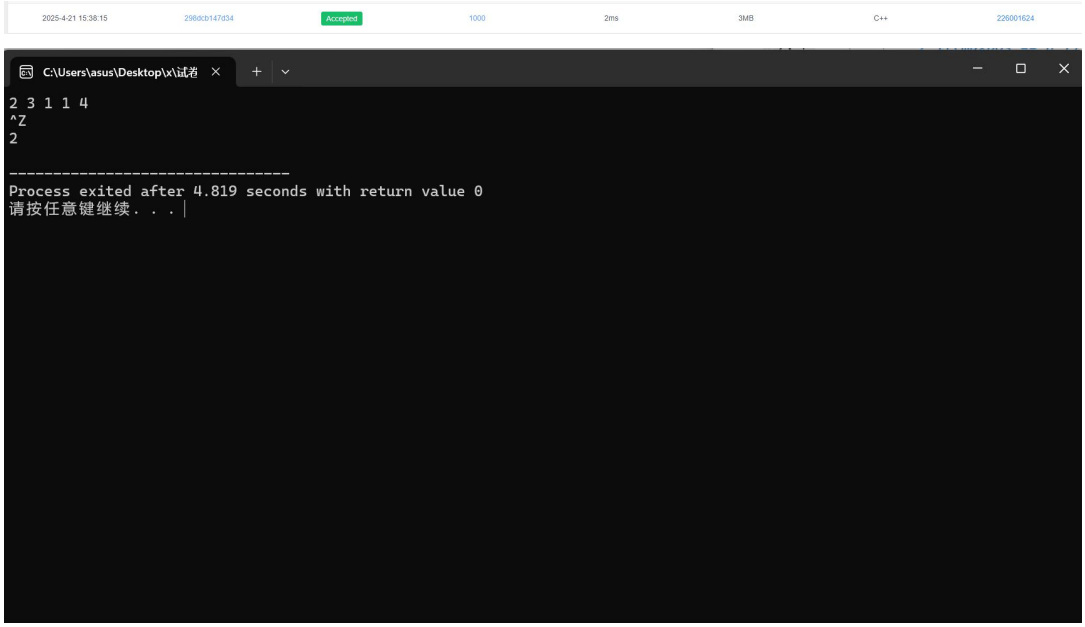
Wrong Answer:在进行比较最后一跳是否能够跳至最后一格 step 其实已经完成了比较，i 的循环无需循环到跳到最后一格的那个位置，因为数组的索引是从 0 开始的，而实际问题的索引是从 1 开始的，所以最终输出的 step 要么开头减一，要么直接最后的值减一。

【4、算法分析与知识点】

根据题目描述，每一步选择中都采取当前状态下最优的选择，从而希望导致结果是全局最优的。在这个跳跃问题中，贪心策略是在每次跳跃时选择能够到达最远位置的步骤进行跳跃。我们需要在每次跳跃范围内寻找在当前跳跃长度内寻找最远的可达位置，并将其位置更新为要选择的位置的索引，接下来只需判断这最后一条是否能够跳至最后一格即可。如此重复以上步骤，得到最少跳跃数。

程序的时间复杂度是：由于遍历整个数组只需要一次线性扫描，因此遍历过程时间复杂度为 O(n)，其中 n 是数组元素的个数。空间复杂度方面，除了输入数组 a 外，仅使用了少量额外变量（n、maxpos、end、step 等），因此空间复杂度为 O(n)，主要是存储输入元素本身。

【5、AC 截图及运行测试截图】



【1、 智力大冲浪（2-1001）】

智力大冲浪

Description

小伟报名参加中央电视台的智力大冲浪节目，本次挑战赛吸引了众多参赛者，主持人为了表彰大家的勇气，先奖励每个参赛者m元。先不要太高兴！因为这些钱还不一定都是你的！接下来主持人宣布了比赛规则：

首先，比赛时间分为n个时段(n≤500),它又给出了很多小游戏，每个小游戏都必须在规定期限ti前完成(1≤ti≤n)。如果一个游戏没能在规定期限前完成，则要从奖励费m元中扣去一部分钱wi，wi为自然数，不同的游戏扣去的钱是不一样的。当然，每个游戏本身都很简单，保证每个参赛者都能在一个时段内完成，而且都必须从整时段开始。主持人只是想考考每个参赛者如何安排组织自己做游戏的顺序。作为参赛者，小伟很想赢得冠军，当然更想赢取最多的钱！注意:比赛绝对不会让参赛者赔钱！

Input

输入文共4行。

第1行为m，表示一开始奖励给每位参赛者的钱；

第2行为n，表示有n个小游戏；

第3行有n个数，分别表示游戏1到n的规定完成期限；

第4行有n个数，分别表示游戏1到n不能在规定期限前完成的扣款数。

Output

输出仅1行，表示小伟能赢取最多的钱.

Sample Input 1

```
10000
7
4 2 4 3 1 4 6
70 60 50 40 30 20 10
```

Sample Output 1

```
9950
```

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int m, n;
bool used[505] = {false}; // 记录每个时间点是否已经安排过

class game {
public:
    int time; // 可安排的最晚时间
    int mon;  // 未完成时的罚金
};

// 按罚金从大到小排序
bool compare(game g1, game g2) {
    return g1.mon > g2.mon;
}

int main() {
    cin >> m >> n;
    game g[n];

    // 输入每个任务的最晚完成时间
    for (int i = 0; i < n; i++) {
        cin >> g[i].time;
    }

    // 输入每个任务对应的罚金
    for (int i = 0; i < n; i++) {
        cin >> g[i].mon;
    }

    sort(g, g + n, compare); // 按罚金排序

    int total = 0; // 累积未完成任务的总罚金

    for (int i = 0; i < n; i++) {
        bool scheduled = false;
        // 从最晚时间往前找空闲时间安排
        for (int t = g[i].time; t >= 1; t--) {
            if (!used[t]) {
                used[t] = true;
                scheduled = true;
                break;
```

```
            }
        }
        // 如果无法安排，累加罚金
        if (!scheduled) {
            total += g[i].mon;
        }
    }

    cout << m - total << endl; // 输出剩余的金币
    return 0;
}
```

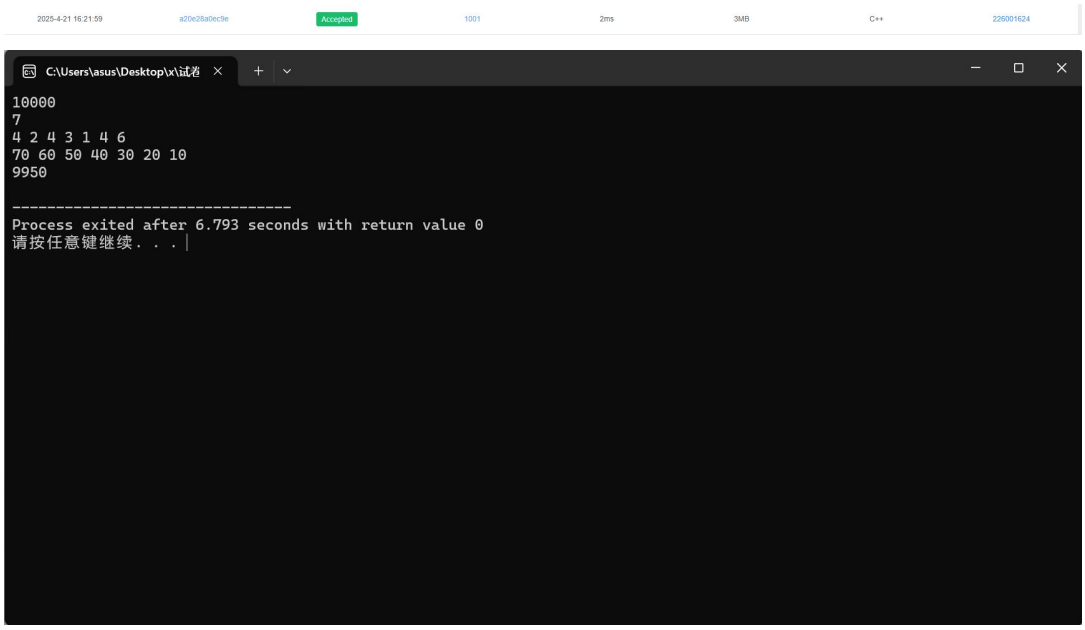
【3、遇到的问题及解决方法】

一开始的想法是把所有活动按时间顺序排好，如果在时间限定范围内够完成，则不做惩罚，若不够，则进行惩罚，但我没有考虑到不同任务的惩罚不同，例如如果两个事件在一个相邻截止日期，那么应该优先考虑惩罚大的，而非严格按照截止日期进行排序，因此实际的排序顺序应该是按照惩罚金额进行排序，然后先把惩罚金额大的在其截止日期安排好。

【4、算法分析与知识点】

根据题目描述，在给定初始金币 m 和 n 个任务的情况下，通过合理安排任务，最大程度地减少因未完成任务而扣除的罚金。每个任务有一个最晚完成时间 time 和对应的罚金 mon，程序采用贪心策略：优先安排罚金最高的任务，尽可能在其截止时间前找一个空闲时间点完成。如果无法安排，则累计该任务的罚金。最终输出剩余金币数，即初始金币 m 减去所有未完成任务的总罚金。程序整体时间复杂度为 O(n log n)，主要由于排序过程；空间复杂度为 O(n)，用于存储任务数据和时间安排情况。

【5、AC 截图及运行测试截图】



【1、删除数字 II（2-1003）】

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

void solve() {
    string num, num1;
    int ans, cnt = 0;

    while (cin >> num >> ans) {
        num1 = num;
        cnt = 0;
        int t;

        // 删除 ans 位数字
        while (cnt < ans) {
            int flag = 0;
            for (int i = 0; i < num.size() - 1; i++) {
                if (num[i] > num[i + 1]) {
                    num.erase(i, 1);
                    cnt++;
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                num.erase(num.size() - 1, 1);
                cnt++;
            }
        }

        int i = 0;
        // 输出被删除的字符（调试信息部分）
        for (int j = 0; j < num1.size(); j++) {
            if (i < num.size() && num[i] == num1[j]) {
                i++;
            } else {
                if (cnt == 1)
                    cout << num1[j] << endl;
                else
                    cout << num1[j] << " ", cnt--;
            }
        }
```

```
42.
43.         // 输出处理后的结果，去除前导零
44.         int idx = 0;
45.         while (idx < num.size() - 1 && num[idx] == '0') idx++;
46.         cout << num.substr(idx) << endl;
47.     }
48.}
49.
50.int main() {
51.     solve();
52.     return 0;
53.}
```

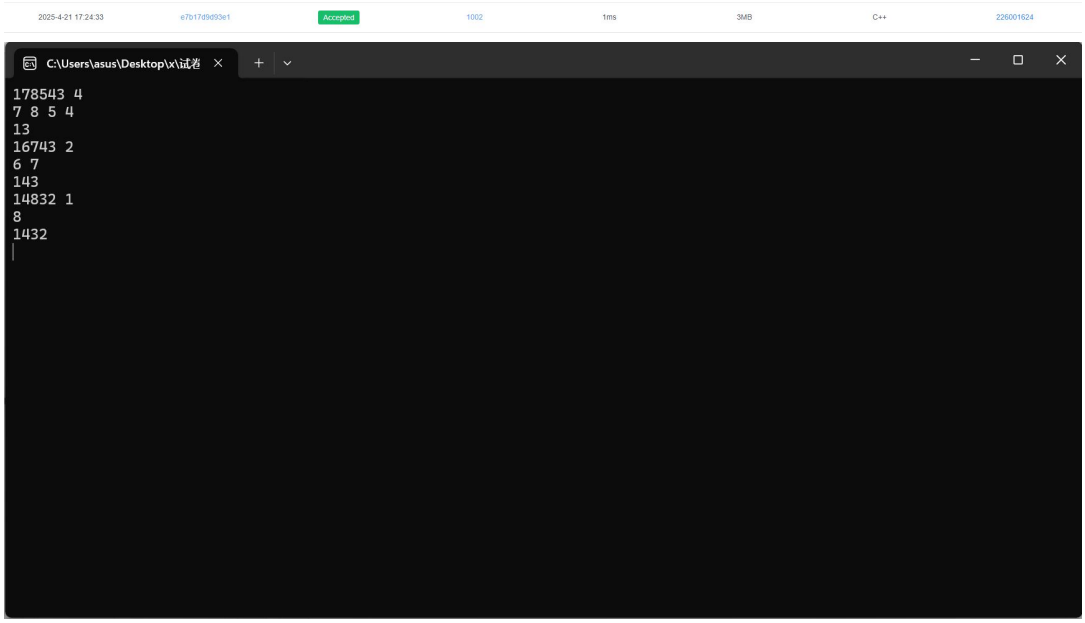
【3、遇到的问题及解决方法】

本题有个前提要求，输出不能有前导零，即 020 之类的需要转化为 20，一开始没注意，因此一直报错，后续加一个判断 if，改一下输出格式即可。

【4、算法分析与知识点】

根据题目描述，我们需要在删除一定数目的数字之后使得字符串保持最小值。这时我们可以假设一下，如果一个数字字符串前端是递增，我们只需要删去刚开始往下递减的哪个数字即可，这样就可以保证较高位的数字较小。相反，若是我们删除了比较靠后的数字，则较高的值依旧比较大，这样就得不到最小的字符串。还有如果一个数字字符串一直保持递增或者是相等的话，我们只需要删除最后一格数字即可。由此我们开始设计我们的算法。首先设置一个外部 while 循环来控制删除操作，直到删除了 ans 个字符串为止，内部 for 循环比哪里数字串，寻找可删除的位置。最后我们再通过 num1 和 num 的比较来输出删除的数字。程序的整体时间复杂度是 $O(n \times ans)$，其中 n 是数字长度；空间复杂度是 O(n)。

【5、AC 截图及运行测试截图】



【1、 汽车加油（2-1003）】

汽车加油

Description

一辆汽车加满油后可行驶 n 公里。旅途中有若干个加油站。设计一个有效算法，指出应在哪些加油站停靠加油，使沿途加油次数最少。

对于给定的 n 和 k 个加油站位置，编程计算最少加油次数。

Input

第一行有 2 个正整数 n 和 k，表示汽车加满油后可行驶n 公里，且旅途中有 k 个加油站。接下来的 1 行中，有 k+1 个整数，表示第 k 个加油站与第k-1 个加油站之间的距离。第 0 个加油站表示出发地，汽车已加满油。第 k+1 个加油站表示目的地。

Output

将计算出的最少加油次数输出; 如果无法到达目的地，则输出 No Solution .

Sample Input 1

```
7 7
1 2 3 4 5 1 6 6
```

Sample Output 1

```
4
```

【2、程序代码及注释】

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> a(k + 1);
    for (int i = 0; i <= k; i++) {
        cin >> a[i];
    }

    int count = 0;      // 加油次数
    int curgas = n;     // 当前油量

    for (int i = 0; i <= k; i++) {
        if (a[i] > n) {           // 路段需求大于油箱容量，无解
            cout << "No Solution" << endl;
            return 0;
        }
        if (curgas >= a[i]) {      // 油够，正常前进
            curgas -= a[i];
        } else {                   // 油不够，加油一次
            count++;
            curgas = n - a[i];
        }
    }
    cout << count << endl;
    return 0;
```

```
30.}
```

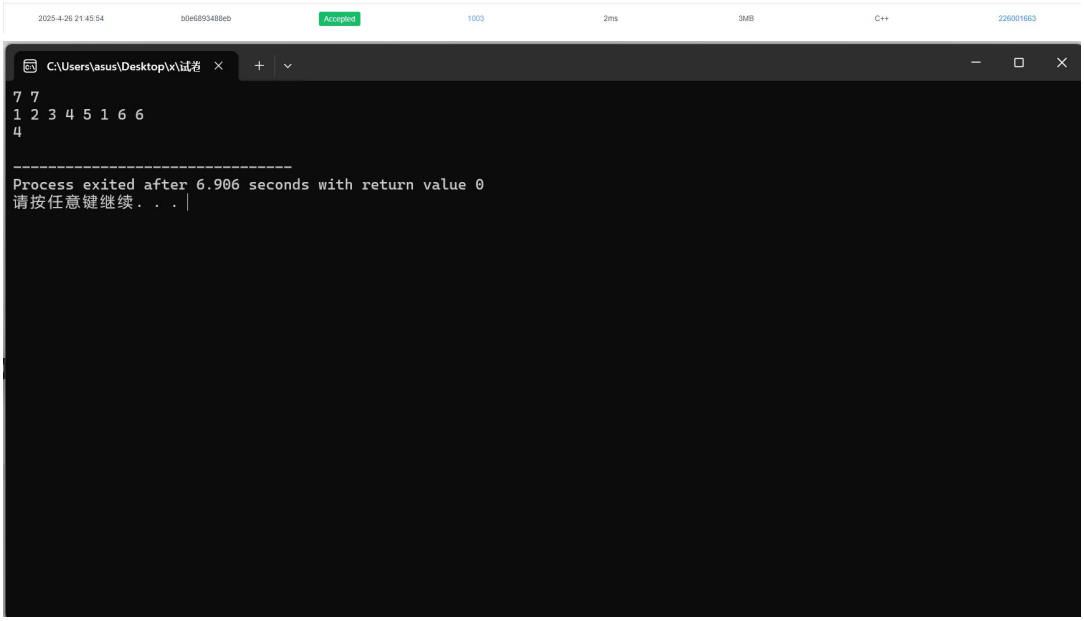
【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

这段程序用于模拟一次长途行驶过程，要求最少加油次数。给定油箱容量 n 和 k+1 个连续路段的耗油量，程序每次判断当前油量是否足够通过当前路段：如果足够则直接通过，否则加满一次油再前进。如果遇到单个路段的耗油量超过油箱容量，则输出 "No Solution"。整体时间复杂度为 O(k)，空间复杂度为 O(k)。

【5、AC 截图及运行测试截图】



Module 3:　贪心算法堂练

【1、　种树（3-1000）】

种树

Description

一条街的一边有几座房子。因为环保原因居民想要在路边种些树，路边的地区被分割成块，并被编号成1..N；每个部分为一个单位尺寸大小并最多可种一棵树，每个居民想在门前种些树并指定了三个号码B,E,T，这三个数表示该居民想在B和E之间最少种T棵树。当然，B≤E,居民必须记住在指定区不能种多于区域地块数的树，所以T≤E-B+l。居民们想种树的各自区域可以交叉。你的任务是求出能满足所有要求的最少的树的数量。

Input

第一行包含数据N，区域的个数；

第二行包含H，房子的数目；

下面的H行描述居民们的需要:B E T。

Output

输出能满足所有要求的最少的树的数。

Sample Input 1

```
9
4
1 4 2
4 6 2
8 9 2
3 5 2
```

Sample Output 1

```
5
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

int N,H;
class peo{
 public:
 int B,E,T;
};

bool compare(peo p1,peo p2){
 return p1.E<p2.E;
}

int main(){
 cin>>N>>H;
 peo peo[H];
 vector<bool> tree(N+1,false);
 for(int i =0;i<H;i++){
  cin>>peo[i].B>>peo[i].E>>peo[i].T;
 }
 sort(peo,peo+H,compare);

 int sum1 = 0;

 for(int i = 0;i<H;i++){
  int cnt = 0;
  for(int j = peo[i].B;j<=peo[i].E;j++){
   if(tree[j]) cnt++;
  }
  int k = peo[i].E;
```

```
31.  while(cnt<peo[i].T){
32.   if(!tree[k]){
33.    tree[k] = true;
34.    cnt++;
35.    sum1++;
36.   }
37.   k--;
38.  }
39. }
40. cout<<sum1<<endl;
41.}
42.
```

【1、 最大摆动序列（3-1001）】



最大摆动序列

Description

如果连续数字之间的差严格地在正数和负数之间交替，则数字序列称为摆动序列。第一个差（如果存在的话）可能是正数或负数。少于两个元素的序列也是摆动序列。

例如，[1,7,4,9,2,5] 是一个摆动序列，因为差值 (6,-3,5,-7,3) 是正负交替出现的。相反，[1,4,7,2,5] 和 [1,7,4,5,5] 不是摆动序列，第一个序列是因为它的前两个差值都是正数，第二个序列是因为它的最后一个差值为零。

给定一个整数序列，通过从原始序列中删除一些（也可以不删除）元素来获得幅度最大的摆动子序列，按原顺序输出该摆动子序列和序列长度。

Input

一个整数数字序列

Output

第1行输出所求摆动子序列；第2行输出摆动子序列的长度.

Sample Input 1

```
1 7 4 9 2 5
```

Sample Output 1

```
1 7 4 9 2 5
6
```

【2、程序代码及注释】

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. void solve() {
5.     vector<int> nums;
6.     int num;
7.     int curDiff,nextDiff;
8.     bool flag = true;
9.     while (cin >> num) {
10.        nums.push_back(num);
11.        if (cin.peek() == '\n') {
12.            cin.ignore();
13.            break;
14.        }
15.    }
16.    int n = nums.size();
17.    if (n <= 1) {
18.        cout << nums[0] <<endl;
```

```
        cout << n << endl;
        return;
    }

    cout<<nums[0];

    int count = 1;

    for (int i = 1; i < n - 1; i++) {
        curDiff = nums[i] - nums[i-1];
        nextDiff = nums[i+1] - nums[i];
        if ((curDiff *nextDiff) < 0) {
            cout <<" "<< nums[i] ;
            count++;
            flag = false;
        }
    }

 if ((curDiff *nextDiff) < 0||flag){
  if(nextDiff!=0){
   cout<<" "<<nums[n-1];
      count++;
  }
 }
    cout << endl;
    cout << count << endl;
}

int main() {
    solve();
    return 0;
}
```

【1、 区间划分（3-1002）】

区间

Description

给出n个区间的起点和终点，求最少使用其中多少个区间可以将所有区间所在的区域完全覆盖。（测试的数据确保这1点）。

Input

第1行一个整数n，表示n个区间；

第2行开始n行，每行2个整数，表示一个区间范围。

类似[1,4][5,6]被认为是覆盖了[1,6]。

Output

从起点开始，按区间先后顺序，输出选中的区间。所选的区间应尽可能向终点扩展。

Sample Input 1

```
7
1 5
1 6
3 6
1 7
6 9
9 10
7 9
```

Sample Output 1

```
1 7
6 9
9 10
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

class inter{
public:
 int start;
 int end;
};

bool compare(inter s1, inter s2) {
 if (s1.start < s2.start)
  return true;
 else if (s1.start == s2.start && s1.end < s2.end)
  return true;
 else
  return false;
}

int n;
int index1 = 0;

int main(){
 cin >> n;
 inter in[n],result[n];
 for(int i= 0;i<n;i++){
  cin>>in[i].start>>in[i].end;
 }
 sort(in,in+n,compare);
 int end1 = in[n-1].end;
 int right = in[0].start-1;
```

```
31.
32. for(int i = 0;i<n-1;){
33.  int maxright = in[i].end,maxindex = i;
34.  while(in[i].start<=right+1&&i<n){
35.   if(in[i].end>maxright){
36.    maxright = in[i].end;
37.    maxindex = i;
38.   }
39.   i++;
40.  }
41.  right = maxright;
42.  i =maxindex;
43.  result[index1++] = in[maxindex];
44.  if(maxright == end1){
45.   break;
46.  }
47. }
48. for(int i = 0;i<index1;i++){
49.  cout<<result[i].start<<" "<<result[i].end<<endl;;
50. }
51.}
```

【第二部分】

【同类题型实验扩展/可选/会影响分值】

1、题目+测试数据+结果

2、分析

3、建议：难度不低于实验题

【1、g3-竞赛分组】

在元旦即将到来之际，校学生会委托乐乐负责新年晚会的纪念品发放工作。为了使参加晚会的同学获得的纪念品价值相对均衡，乐乐需要将购来的纪念品根据价格进行分组。每组最多只能包含两件纪念品，并且每组纪念品的价格之和不能超过一个给定的整数。为了在尽量短的时间内发完所有纪念品，乐乐希望分组的数目最少。

Google Sites

输入格式：

- 第一行：一个整数 `w`，表示每组纪念品价格之和的上限。
- 第二行：一个整数 `n`，表示购来的纪念品的总件数。
- 接下来的 `n` 行：每行一个正整数 `p_i`，表示第 `i` 件纪念品的价格，满足 `5 <= p_i <= w`。

Google Sites +1

输出格式：

- 仅一行，包含一个整数，表示最少的分组数目。 Google Sites +2

【2、程序代码及注释】

```
#include<iostream>
#include<algorithm>
using namespace std;
int num[30010];  //定义数组用于存储学生成绩
int main(){
    int w,n,ans=0; //w 为成绩合上限，n 为学生数，ans 为分组数
    cin>>w>>n;   //输入数据
    for(int i=0;i<n;i++)
        cin>>num[i];
    sort(num,num+n); //按学生成绩进行排序
    int i=0,j=n-1; //初始化两个指针，一个指向成绩最低的学生，一个指向成绩最高的学生
    while (i<=j){ //遍历所有学生
        if(num[j]+num[i]<=w){ //判断二者之和是否超过上限
            ans++; //若未超过则分组数增加，两个指针同时向中间移动
            j--;
            i++;
        }else{  //若超过则仅仅只移动成绩高的指针，分组数增加
            ans++;
            j--;
        }
    }
    cout<<ans<<endl;
    return 0;
}
```

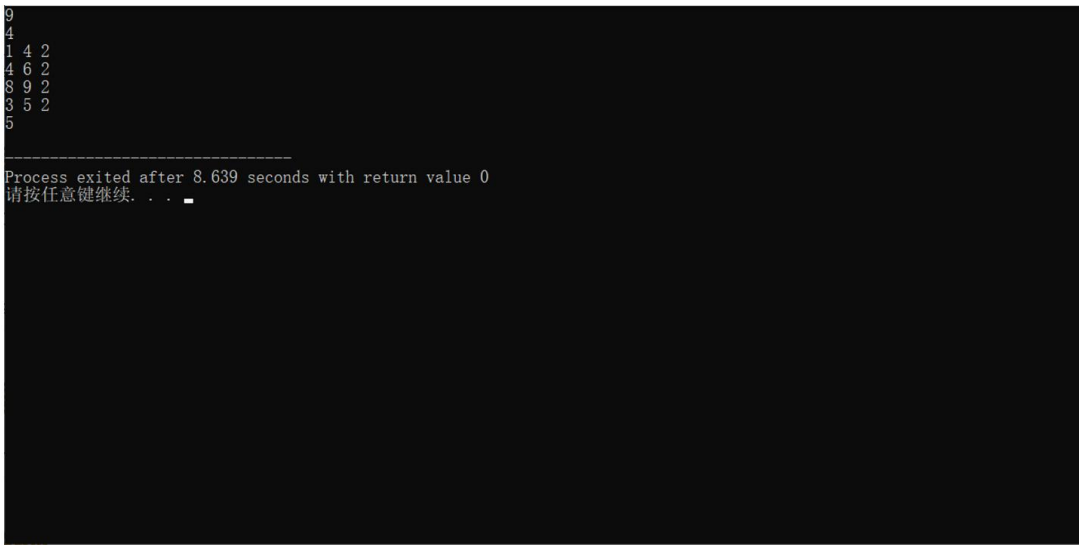
【3、遇到的问题及解决方法】

Wrong Answer:在原先的算法中，在判断边界条件时产生错误，我原先的想法是比较到 while (i<j)，最后在返回时将 ans 执行+1 操作，但这样做只考虑到剩余个数为 1 的情况，未考虑到剩余的数目为二且两者恰好可以组队的情况。

【4、算法分析与知识点】

根据题目描述，我们需要在满足成绩合不超过上限的情况下选择出最少的组数。首先对输入的成绩由小到大进行排序，接着使用两个指针 i 和 j 分别指向数组的最低分和最高分。通过调整这两个指针位置，尝试将成绩组合成不超过上限 w 的组。每次尝试将指针 i 和 j 所指向的成绩相加，如果总和不超过上限，则将这两个成绩视为一组，并将两个指针向中间移动，如果超过上限，则只移动 j 指针，开始新的一组。

【5、AC 截图及运行测试截图】



【1、买股票的最佳时机】

122. 买卖股票的最佳时机 II

已解答

中等 相关标签 相关企业 Aa

给你一个整数数组 `prices` ，其中 `prices[i]` 表示某支股票第 `i` 天的价格。

在每一天，你可以决定是否购买和/或出售股票。你在任何时候 **最多** 只能持有 **一股** 股票。你也可以先购买，然后在 **同一天** 出售。

返回 *你能获得的* ***最大*** *利润* 。

示例 1：

> **输入：** prices = [7,1,5,3,6,4]
> **输出：** 7
> **解释：** 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。
> 随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，这笔交易所能获得利润 = 6 - 3 = 3。
> 最大总利润为 4 + 3 = 7 。

示例 2：

> **输入：** prices = [1,2,3,4,5]
> **输出：** 4
> **解释：** 在第 1 天（股票价格 = 1）的时候买入，在第 5 天 （股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。
> 最大总利润为 4 。

示例 3：

> **输入：** prices = [7,6,4,3,1]
> **输出：** 0
> **解释：** 在这种情况下，交易无法获得正利润，所以不参与交易可以获得最大利润，最大利润为 0。

【2、程序代码及注释】

```
1. class Solution {
2. public:
3.     // 主函数，输入价格数组 prices，返回最大利润
4.     int maxProfit(vector<int>& prices) {
5.         int n = prices.size();  // 获取价格数组的大小
6.         vector<int> dp(n, 0);  // dp[i] 表示到第 i 天为止能获得的最大利润
7.         int minprice = prices[0], index1 = 0;  // 初始化最小价格为第0天的价格，index1 为最小价格的日期
8.         dp[0] = 0;  // 第0天不能获得任何利润，因此 dp[0] = 0
9.
10.        // 遍历从第1天到最后一天
11.        for(int i = 1; i < n; i++) {
12.            if(prices[i] < prices[i-1]) {  // 价格发生下降，意味着需要卖出
13.                dp[i] = dp[i-1];  // 如果股价下跌，则当前利润不变
14.                minprice = prices[i];  // 更新最小价格为当前价格
15.                index1 = i;  // 更新买入的日期
16.            } else {  // 价格上涨，更新最大利润
17.                dp[i] = dp[index1] + prices[i] - minprice;  // 计算卖出时机所得到的最大利润
18.            }
19.        }
```

```
20.
21.         return dp[n-1];  // 返回到最后一天的最大利润
22.     }
23. };
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

首先，初始化 minprice 为第 0 天的股价，表示当前为止最低的股价，并设定 dp[0] = 0（因为没有交易时利润为 0）。然后，遍历从第 1 天开始的所有价格，检查是否股价下跌。如果股价下跌，则意味着要卖出，并更新 minprice 和 index1（记录最小价格的日期）。如果股价上涨，则计算从 index1 日期开始的买入和当前日期卖出的利润，更新最大利润。

时间复杂度为 O(n)，因为只需要一次遍历所有价格。空间复杂度为 O(n)，由于使用了一个 dp 数组来记录每一天的最大利润。如果我们优化空间，只使用常数空间来存储最大利润和最低股价，空间复杂度可以降低到 O(1)。

【5、AC 截图及运行测试截图】



【1、加油站】

134. 加油站

已解答

中等 相关标签 相关企业 A文

在一条环路上有 `n` 个加油站，其中第 `i` 个加油站有汽油 `gas[i]` 升。

你有一辆油箱容量无限的的汽车，从第 `i` 个加油站开往第 `i+1` 个加油站需要消耗汽油 `cost[i]` 升。你从其中的一个加油站出发，开始时油箱为空。

给定两个整数数组 `gas` 和 `cost` ，如果你可以按顺序绕环路行驶一周，则返回出发时加油站的编号，否则返回 `-1` 。如果存在解，则 **保证** 它是 **唯一** 的。

示例 1:

> **输入:** gas = [1,2,3,4,5], cost = [3,4,5,1,2]
> **输出:** 3
> **解释:**
> 从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油
> 开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油
> 开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油
> 开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油
> 开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油
> 开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。
> 因此，3 可为起始索引。

【2、程序代码及注释】

```
1. class Solution {
2. public:
3.     int canCompleteCircuit(vector<int>& gas, vector<int>& cost)
    {
4.         int ans = 0, min_s = 0, s = 0;  // s 表示油量，min_s 表示
   最小油量
5.         for (int i = 0; i < gas.size(); i++) {
6.             s += gas[i] - cost[i]; // 在 i 处加油，然后从 i 到 i+1
7.             if (s < min_s) {
8.                 min_s = s; // 更新最小油量
9.                 ans = i + 1; // 注意 s 减去 cost[i] 之后，汽车
   在 i+1 而不是 i
10.            }
11.        }
12.        // 循环结束后，s 即为 gas 之和减去 cost 之和
13.        return s < 0 ? -1 : ans;
14.    }
15.};
```

【3、遇到的问题及解决方法】

一开始通过遍历+剪枝，结果还是超时了。。。。

【4、算法分析与知识点】

这段代码是通过模拟加油站问题来确定一个合适的起始点，使得从该点出发能够绕一圈并回到起点。通过遍历每个加油站，计算从当前站点出发时的油量变化，判断是否能够成功完成环绕一圈。具体来说，变量 s 用来跟踪当前油量，min_s 用来记录过程中油量的最小值，ans 则记录当前能完成一圈的起始站点。每次当油量小于 min_s 时，更新起始站点为下一个站点。最终，如果总油量 s 小于 0，说明无法完成旅行，返回 -1；否则返回找到的起始站点。该算法的时间复杂度为 O(n)，空间复杂度为 O(1)，因此它是一个高效的解决方案。

【5、AC 截图及运行测试截图】

